Recursion

1) Break your problem into smaller:

faith

this will work

2) Do the work to reach your faith

3) Think of the smallest problem.

$n \to 1$

$n = 5$

```
fun pD (int n){
    print(n);
    pd(n-1);
}
```

Expectation

It will print numbers from n to 1.

$n = 5$

5
4
3
2
1

work

Faith (smaller)

It will print numbers from (n-1 to 1.)

4
3
2
1

power of x^y

$(x, y) \Rightarrow x^y$

psv int pow (int x, int y){

int sA= pow (x, y -1);

int ans = x × sA;

}

Faith (smaller)

It will return

x raised to (y-1)

2^3

2^3

$2 \times 2 \times 2$

2^4 $x=2$ $y=4$

Expectation

It will return

x raised to

power y.

2^4

$2 \times 2 \times 2 \times 2$

$y = 0$

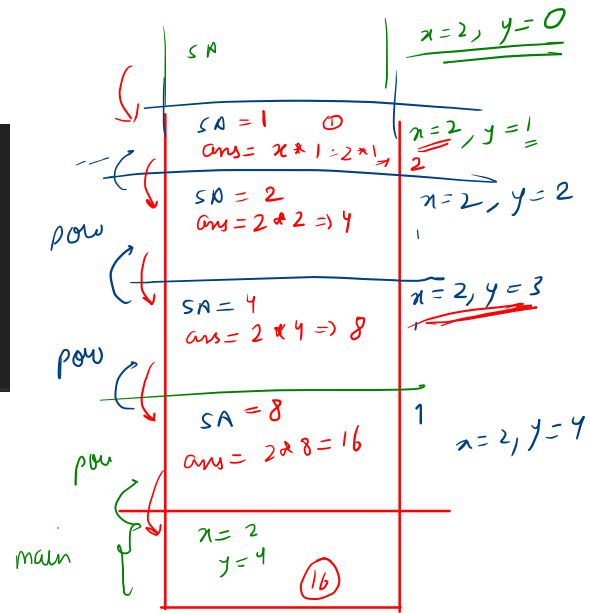
ans = 1

```
public static int pow(int x, int y){

    1) int smallerAns = pow(x,y-1);
    2) int ans = x * smallerAns;

    3) return ans;
}
```

if (y==0){

return 1;

}



Friends Pairing

n ways single => (n-1) ways

pair up

(n-1) single

(n-2) pair up

1, 2, 3, 4

n-1 people

n=3 — 4 ways

1, 2, 3,

{1,2}, 3

{1,3} 2

{2,3}, 1

n=4 — 10 ways

1, 2, 3, 4

{1,2}, 3, 4

{1,3}, 2, 4

{1,4} 2, 3

1, {2,3} 4

1, {2,4}, 3

1, 2, {3,4}

{1,2} {3,4}

{1,4} {2,3}

{1,3} {2,4}

psv int fp (int n) {

n-2

n-2

2

n-2

n-1

3

(n-1) * f(n-2) (4,1) → 2,3 / {2,3} => (4,1), 2,3 ; (4,1) (2,3)

(n-1) * f n (n-2)

n-1 pairs

(4,2) → 1,3 (1,3) / {1,3} => (4,2) 1,3 ; (4,2) (1,3)

(4,3) → 1,2 / {1,2} => (4,3) 1,2 ; (4,3) (1,2)

A	B	C	D

ABC

E → single (10)

→ pairing (n-1) * fn(n-2)

(A,E)

n-1 ← (1, 5) → 2,3,4 ; 2,3,4 ; (2,3), 4 ; (2,4) 3 ; 2, (3 4)

Expectation

It will give us total number of ways for n

Faith

1, 2, 3

```java
public static int friendPairing(int n){
    int single = friendPairing(n-1);
    int pairUp = (n-1) * friendPairing(n-2);

    int totalWays = single + pairUp;

    return totalWays;
}
```

n = 4

if (n == 1) return 1;

if (n == 2) return 2;

if (n <= 2) return n;

s = 2

pU = 2 * fp(n-2)

⇒ 2 * 1 ⇒ 2

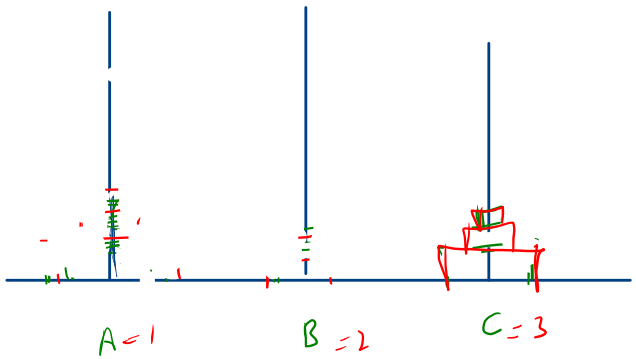
pU = (4-1) * 2

⇒ 3 × 2

⇒ 6

Tower of Hanoi

n=3



toh(int n, int A, int C, int B){

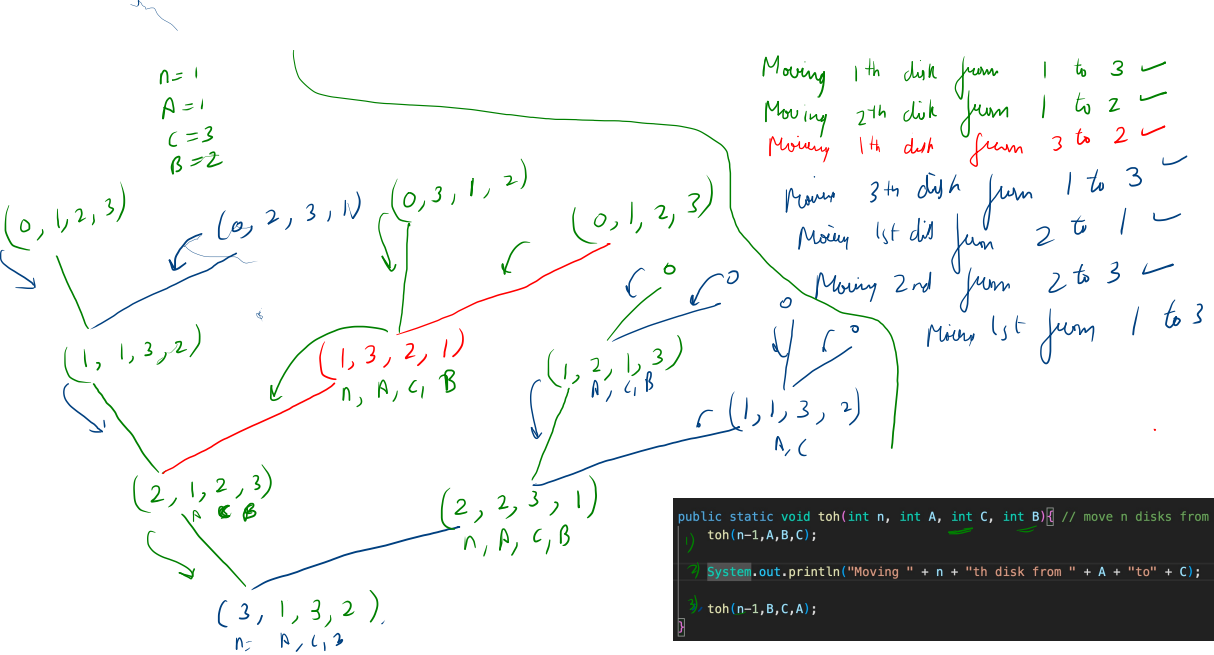
}

→ no bigger disk should be on top of smaller disk.

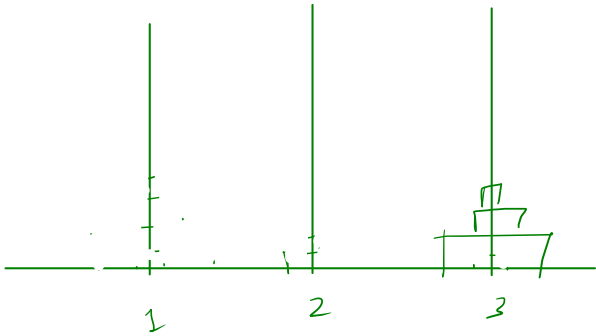
→ pick one disc at one time.

3 → (A, C) with help of B

```
public static void toh(int n, int A, int C, int B){ // move n disks from A to C using B
    toh(n-1,A,B,C);

    System.out.println("Moving " + n + "th disk from " + A + "to" + C);

    toh(n-1,B,C,A);
}
```



Moving 1th disk from 1 to 3
Moving 2th disk from 1 to 2
Moving 1th disk from 3 to 2
Moving 3th disk from 1 to 3
Moving 1st disk from 2 to 1
Moving 2nd from 2 to 3
Moving 1st from 1 to 3




```java
public static void toh(int n, int A, int C, int B){ // move n disks from A to C using B
    toh(n-1,A,B,C);

    System.out.println("Moving " + n + "th disk from " + A + "to" + C);

    toh(n-1,B,C,A);
}
```